

## 1. État des Lieux : Ce qui est fait vs Ce qui reste à faire

### ✅ Ce qui est fait (Validé)

1. **Environment** : Laravel 12 installé.
2. **Tech Basics** : Socialite (OAuth) fonctionne techniquement.
3. **Connexion** : CORS configuré, Nuxt parle à Laravel.

### ⚠️ Ce qui reste à faire (Priorité Absolue)

1. **Architecture BDD** : Créer le schéma relationnel complet (Users <-> Services <-> AREAs).
2. **Service Interface** : Définir une interface PHP stricte pour que le Moteur puisse parler à n'importe quel Service (Google, Discord, etc.) sans changer le code.
3. **Refresh Tokens** : Gérer l'expiration des tokens OAuth (sinon l'automatisation casse après 1h).
4. **Documentation API** : Générer une doc (Swagger/Postman) pour ne pas bloquer les équipes Frontend.

## 2. Le Contrat d'Interface (API pour le Frontend)

**Règle d'or** : Toutes les réponses d'erreur doivent suivre ce format :

```
{ "message": "Description humaine", "errors": { "field": ["detail"] } }
```

### A. Authentication & User

- POST /api/auth/register : {name, email, password, password\_confirmation}.
- POST /api/auth/login : {email, password} -> Retourne { token: "..."}.
- GET /api/user : Retourne l'objet user.

### B. Services (Liaison)

- GET /api/services : Liste les services et l'état de connexion.
  - Réponse : [{ "id": "google", "name": "Google", "connected": true, "icon": "url..." }]
- GET /api/services/{service}/connect : Retourne { "url": "<https://accounts.google.com/>..." }.
- GET /api/services/{service}/callback : Gère le retour OAuth (ne retourne rien de visuel, redirige vers le Front).

- DELETE /api/services/{service} : Supprime le token en base.

### C. AREAs (CRUD)

- GET /api/areas : Liste les automatisations.
- POST /api/areas : Créer.
  - Body : { "service\_action\_id": "google", "action\_id": "new\_mail", "service\_reaction\_id": "discord", "reaction\_id": "send\_message", "parameters": { ... } }
- DELETE /api/areas/{id}.
- PUT /api/areas/{id}/toggle : Active/Désactive.

### D. Système

- GET /about.json : **Format strict du PDF**. Doit inclure client.host (IP de celui qui fait la requête) et server.current\_time.

## 3. Répartition des Tâches Optimisée

### Retys (Architecte, Infra & Engine)

Focus : Le "Squelette" et le "Cerveau".

1. **Migrations BDD** : Crée les tables services, user\_services (pour les tokens), areas (pour les liens), area\_logs.
2. **Le Contrat (Interface)** : Crée l'interface PHP ServiceInterface que Asaph devra implémenter.
  - a. Exemple : public function checkTrigger(array \$params): bool;
3. **L'Engine** : Code la commande php artisan area:execute qui :
  - a. Récupère toutes les AREAs actives.
  - b. Instancie le bon Service.
  - c. Vérifie le Trigger -> Si Oui -> Exécute la Réaction.

### Méryl (Auth, API Management & Conformité)

Focus : La "Porte d'entrée" et la "Documentation".

1. **API Docs** : Installe **Scramble** (package Laravel) ou maintient un Postman à jour pour le Front. C'est toi qui parles aux équipes Front.
2. **Route /about.json** : Implémente la logique dynamique (boucle sur les services disponibles).

3. **CRUD Areas** : Code les contrôleurs pour Créer/Lister/Supprimer les AREAs (c'est ce que le Front va appeler).

### **Asaph (Intégrateur Services & OAuth)**

Focus : Les "Muscles" et "Connexions".

1. **Service Google Class** : Implémente l'interface définie par Retys.
  - a. Trigger : NewGmailReceived.
  - b. Reaction : SendEmail.
2. **Gestion Token** : C'est ta priorité. Quand tu appelles Google, si ça échoue (401), tu dois utiliser le refresh\_token pour en avoir un nouveau et mettre à jour la BDD.
3. **Service Timer** : Implémente un service "Timer" simple (Action : "Every Minute") pour permettre à Retys de tester son moteur sans avoir besoin de Google.

## 4. Roadmap Technique Détaillée

### **Semaine 1 : Fondations Solides (Urgent)**

- **Lundi** : Retys migre tout sur MariaDB (Docker). Méryl finit Login/Register.
- **Mardi** : Retys livre les Migrations (Tables). Asaph livre le "Service Timer".
- **Mercredi** : Méryl livre la doc API (Swagger/Postman) pour les Fronts.
- **Jeudi** : Asaph finit la logique de stockage des Tokens OAuth (Table user\_services).
- **Vendredi** : **CI/CD en place** (Voir section 5).

### **Semaine 2 : Le Moteur & Google**

- **Lundi** : Retys code la commande area:execute (boucle infinie). Il teste avec le Service Timer d'Asaph.
- **Mercredi** : Asaph livre le GoogleService complet (Trigger + Réaction + Refresh Token).
- **Jeudi** : Méryl connecte le Controller de création d'AREA à la BDD.
- **Vendredi** : **Premier test End-to-End** : On crée une AREA "Timer -> Log" via l'API, et le moteur l'exécute.

### **Semaine 3 : Consolidation & MVP**

- **Lundi** : Test End-to-End : "Google Mail -> Action".
- **Mercredi** : Nettoyage du code, ajout de logs pour le débogage.
- **Vendredi** : Code Freeze pour le MVP.

## 5. Pour valider l'Objectif 1.2 (CI/CD) - À faire tout de suite

Copiez ce fichier, il est propre et fonctionnel pour Laravel. Il lance les tests à chaque push.

**Fichier :** .github/workflows/laravel.yml

code Yaml

downloadcontent\_copy

expand\_less

name: Laravel CI

on:

push:

branches: [ "main", "develop" ]

pull\_request:

branches: [ "main", "develop" ]

jobs:

laravel-tests:

runs-on: ubuntu-latest

defaults:

run:

working-directory: ./backend-area # <--- IMPORTANT : Adaptez au nom de votre dossier

steps:

- uses: actions/checkout@v3

- name: Setup PHP

uses: shivammathur/setup-php@v2

with:

php-version: '8.2'

extensions: mbstring, dom, fileinfo, mysql

- name: Copy .env

run: php -r "file\_exists('.env') || copy('.env.example', '.env');"

- name: Install Dependencies

run: composer install -q --no-ansi --no-interaction --no-scripts --no-progress --prefer-dist

- name: Generate key

run: php artisan key:generate

- name: Directory Permissions  
run: chmod -R 777 storage bootstrap/cache

# On utilise SQLite en mémoire pour les tests CI (c'est plus rapide que de lancer MariaDB dans GitHub Actions)

- name: Create Database  
run: |  
 mkdir -p database  
 touch database/database.sqlite

- name: Execute tests (Unit and Feature)  
env:  
 DB\_CONNECTION: sqlite  
 DB\_DATABASE: database/database.sqlite  
run: php artisan test

**Action immédiate :** Commitez ce fichier dès maintenant. Dès que le rond vert apparaît sur GitHub, vous avez validé un point du barème.